

- **سوال ۱:** اسکریپتی بنویسید که دقیقاً یک آرگومان (نام یک فایل) را بپذیرد. اگر تعداد آرگومان‌ها صفر یا بیشتر از یک بود، پیغام خطا چاپ کرده و با کد خروج ۱ از اسکریپت خارج شود (exit 1). اگر آرگومان درست بود، چک کند که آیا آن فایل وجود دارد یا نه و پیغام مناسب را چاپ کند.

```
#!/usr/bin/env bash

if [ $# -ne 1 ]; then
    echo "Error: exactly one argument (file name) is required."
    exit 1
fi

file="$1"

if [ -e "$file" ]; then
    echo "File exists: $file"
else
    echo "File does not exist: $file"
fi
```

تعداد آرگومان‌ها

1-ne یعنی مساوی 1 نیست

```
if [ $# -ne 1 ]; then
```

```
echo "Error: exactly one argument (file name) is required."
```

```
exit 1
```

```
fi
```

اگر تعداد آرگومان‌ها دقیقاً 1 نباشد، وارد این بخش می‌شود

یعنی اسکریپت با خطا تمام شد

در bash ، خروجی 0 معمولاً موفقیت و غیر صفر معمولاً خطا را نشان می‌دهد

- **سوال ۱:** اسکریپتی بنویسید که دقیقاً یک آرگومان (نام یک فایل) را بپذیرد. اگر تعداد آرگومان‌ها صفر یا بیشتر از یک بود، پیغام خطا چاپ کرده و با کد خروج ۱ از اسکریپت خارج شود (`exit 1`). اگر آرگومان درست بود، چک کند که آیا آن فایل وجود دارد یا نه و پیغام مناسب را چاپ کند.

\$1 اولین آرگومان است

چون فقط یک آرگومان مجاز است، همان را به متغیر `file` می‌ریزیم

`file="$1"`

- **سوال ۱:** اسکریپتی بنویسید که دقیقاً یک آرگومان (نام یک فایل) را بپذیرد. اگر تعداد آرگومان‌ها صفر یا بیشتر از یک بود، پیغام خطا چاپ کرده و با کد خروج ۱ از اسکریپت خارج شود (`exit 1`). اگر آرگومان درست بود، چک کند که آیا آن فایل وجود دارد یا نه و پیغام مناسب را چاپ کند.

آیا فایلی که داری میگی، وجود داره یا نه؟

```
if [ -e "$file" ]; then
```

```
    echo "File exists: $file"
```

```
else
```

```
    echo "File does not exist: $file"
```

```
fi
```

- **سوال ۲:** چرا در شرط ["\$VAR" = "\$value"] استفاده از دابل کوتیشن دور متغیر (\$VAR) یک عادت خوب و ضروری است؟ اگر متغیر \$VAR خالی یا حاوی فاصله باشد و کوتیشن نگذاریم چه اتفاقی می افتد؟

کوتیشن دابل برای متغیرها در شرطها لازم است تا :

- (a) **متغیر خالی** شرط را خراب نکند
- (b) **متغیر دارای فاصله** به چند کلمه شکسته نشود
- (c) **اسکرپت قابل اعتماد** و ایمن بماند

- سوال ۲: چرا در شرط ["\$VAR" = "\$value"] استفاده از دابل کوتیشن دور متغیر (\$VAR) یک عادت خوب و ضروری است؟ اگر متغیر \$VAR خالی یا حاوی فاصله باشد و کوتیشن نگذاریم چه اتفاقی می افتد؟

اگر کوتیشن نذاریم چه میشه؟

VAR=""

اگر متغیر خالی باشه

value="Melon_🍉"

و بنویسیم :

if [\$VAR = "\$value"]; then

Convert to

if [= "Melon_🍉"]; then

و این در Bash ، Expression error میده چون یک طرف شرط از بین رفته

- سوال ۲: چرا در شرط `["$VAR" = "$value"]` استفاده از دابل کوتیشن دور متغیر (`$VAR`) یک عادت خوب و ضروری است؟ اگر متغیر `$VAR` خالی یا حاوی فاصله باشد و کوتیشن نگذاریم چه اتفاقی می افتد؟

اگر کوتیشن نذاریم چه میشه؟

`VAR="hello kade"`

اگر متغیر فاصله داشته باشه

`value="hello kade"`

اگر بدون کوتیشن بنویسیم :

`if [ $VAR = "$value" ]; then`

Bash breaks it down like this

`if [ hello kade = "hello kade" ]; then`

اینجا `hello kade` به دو آرگومان تبدیل میشه و شرط خراب میشه

- **سوال ۲:** چرا در شرط `["$VAR" = "$value"]` استفاده از دابل کوتیشن دور متغیر (`$VAR`) یک عادت خوب و ضروری است؟ اگر متغیر `$VAR` خالی یا حاوی فاصله باشد و کوتیشن نگذاریم چه اتفاقی می افتد؟

چرا دابل کوتیشن این مشکلات را حل می‌کند؟

```
if [ "$VAR" = "$value" ]; then
```

✓ مقدار کامل `VAR` یکجا بررسی میشود

✓ اگر خالی هم باشد، به صورت `""` باقی میماند

✓ اگر فاصله داشته باشد، باز هم یک رشته ی واحد است

- **سوال ۳:** با استفاده از حلقه `for`، اسکریپتی بنویسید که تمام فایل‌های موجود در دایرکتوری فعلی را لیست کرده و برای هر کدام مشخص کند که فایل است یا دایرکتوری.

```
#!/usr/bin/env bash
```

```
for item in *; do  
    if [ -f "$item" ]; then  
        echo "$item is a file"  
    elif [ -d "$item" ]; then  
        echo "$item is a directory"  
    fi  
done
```

- **سوال ۳:** با استفاده از حلقه `for`، اسکریپتی بنویسید که تمام فایل‌های موجود در دایرکتوری فعلی را لیست کرده و برای هر کدام مشخص کند که فایل است یا دایرکتوری.

```
for item in *; do
    if [ -f "$item" ]; then
        echo "$item is a file"
    elif [ -d "$item" ]; then
        echo "$item is a directory"
    fi
done
```

همه‌ی فایل‌ها و پوشه‌های داخل دایرکتوری فعلی را برمی‌گردونه

هر بار یکی از آن‌ها در متغیر `item` قرار می‌گیره

آیا این آیتم یک فایل معمولیه ؟

آیا این آیتم یک دایرکتوری هست ؟